

Runbook: How to Run the Project

From an empty machine to the live dashboard

Olist delivery-performance pipeline

Repository: github.com/PeterLees/bigData-module2-project-group7

Companion to: Module2_Group7_Development_Plan.docx

STEPS 1 TO 5 ARE DONE ONCE. AFTER THAT, ONLY STEPS 6 AND 7 ARE NEEDED.

Follow the steps in order. Each one ends with a verification command, and you should not move on until it passes.

Read Section 1 first: it says which steps only one person performs, and which every member must run on their own machine.

Contents

1. Before You Start — who does what; the two first-run fixes
2. Step 1 — Python Environment
3. Step 2 — Create the GCP Project — Platform Owner only
4. Step 3 — Authenticate Locally — every member
5. Step 4 — Download the Dataset
6. Step 5 — Bucket, Datasets and Config
7. Step 6 — Run the Pipeline
8. Step 7 — Open the Dashboard
9. Optional Outputs
10. Troubleshooting
11. What Has and Has Not Been Verified

1. Before You Start

This runbook takes a machine with nothing installed and ends with the interactive dashboard open in a browser. It assumes you have cloned the repository and can open a terminal in it.

1.1 Who does what, and how often

Step	What	Who runs it	How often
1	Create the Python environment	Every member	Once per machine
2	Create the GCP project, billing, APIs, team access	Platform Owner only	Once for the team
3	Authenticate gcloud locally	Every member	Once per machine
4	Download the Olist dataset from Kaggle	Whoever will run the load	Once
5	Create the bucket, the datasets and the config files	Platform Owner creates the shared datasets; every member creates their own dev dataset and config	Once
6	Run the pipeline: <code>make all</code>	Anyone	Every time data or models change
7	Open the dashboard: <code>make dashboard</code>	Anyone	Any time

1.2 Two things that were fixed for the first run

Both were real defects that would have stopped the very first run. They are already corrected in the repository; this note exists so the team understands why the Makefile looks the way it does.

What was wrong	How it is handled now
dbt could not see .env. python-dotenv loads <code>.env</code> for the Python scripts, but dbt resolves <code>env_var()</code> against the real process environment, so every dbt target failed with <i>"Env var required but not provided: GCP_PROJECT_ID"</i> .	The Makefile now includes and exports <code>.env</code> automatically, so every target sees it. Nothing to remember and nothing to type.
make all ran the snapshot before the models. The snapshot reads <code>stg_products</code> , a view that does not exist yet on a fresh warehouse, so the first run always failed. The step was also redundant, because <code>dbt build</code> already runs snapshots in dependency order.	The separate snapshot step was removed from the pipeline. <code>dbt build</code> handles it in the correct order. The same ordering bug was fixed in the nightly GitHub Actions workflow.

2. Step 1 — Python Environment

Every member. Approximately 10 minutes.

PYTHON VERSION MATTERS. The system default python3 on this machine is 3.14, which dbt does not support. The project is pinned to Python 3.11. Installing into the wrong interpreter is the single most common way this setup goes wrong, and the error it produces is confusing.

```
conda create -y -n bd-m2-g7 python=3.11
```

```
conda activate bd-m2-g7
```

```
cd /path/to/bigData-module2-project-group7  
make setup
```

`make setup` installs `requirements.txt` and then runs `dbt deps` to fetch the `dbt_utils` and `dbt_expectations` packages.

Verify:

```
python --version
```

Must print Python 3.11.x. If it prints 3.14, the conda environment is not active.

3. Step 2 — Create the GCP Project

Platform Owner only, once for the whole team. Approximately 15 minutes.

1. Open <https://console.cloud.google.com> and sign in with the Google account that will own the project. Use an account you will still control at submission time.
2. **Enable billing.** In the left menu choose Billing, then “Link a billing account”. A new Google Cloud account receives a 300 USD, 90-day free trial. Billing has to be enabled for the BigQuery API to function at all, but the always-free tier (1 TB of query scan and 10 GB of storage per month) covers this entire project many times over, because the dataset is only about 120 MB. **Expected real spend: zero.**
3. **Create the project.** Click the project selector at the top of the console, then “New Project”. Name it `bigdata-m2-group7`. Google appends digits to make a globally unique **Project ID**, for example `bigdata-m2-group7-473012`. Copy that Project ID: it is *not* the same as the display name, and every member needs it.
4. **Enable the APIs.** Go to “APIs and Services”, then “Enable APIs and Services”. Search for and enable the **BigQuery API**, then do the same for the **Cloud Storage API**. Each takes about 30 seconds to activate.
5. **Invite the team.** Go to “IAM and Admin”, then “IAM”, click “Grant access”, and add each teammate’s Google email with the role **BigQuery Job User**. This lets them run queries but not delete datasets. Finer dataset-level permissions are granted in Step 5.

Verify:

```
gcloud projects describe <PROJECT_ID>
```

The output must include `lifecycleState: ACTIVE`. Then post the Project ID in the team chat, because everyone needs it in Step 5.

4. Step 3 — Authenticate Locally

Every member, on their own machine. Approximately 5 minutes.

Run these three commands in order. The first two open a browser window; approve with the same Google account that was invited in Step 2.

```
gcloud auth login
```

Grants the `gcloud` and `bq` command-line tools access to your account.

```
gcloud auth application-default login
```

Creates the **Application Default Credentials** that `dbt`, the Python client library, `SQLAlchemy` and the dashboard all read. This is a *separate* credential from the one above. Skipping it is the number one cause of “*could not automatically determine credentials*” later on.

```
gcloud config set project <PROJECT_ID>
```

Verify:

```
bq ls
```

This must return without an authentication error. It will list the datasets once Step 5 is done.

NEVER DOWNLOAD A SERVICE-ACCOUNT KEY FOR LOCAL WORK. User OAuth makes every BigQuery job attributable to a person, which is what the course access-control guidance requires. The only service account in this project is for GitHub Actions, and its key lives solely in GitHub Secrets. A leaked service-account key in a public repository is an immediate and serious finding.

5. Step 4 — Download the Dataset

Whoever will run the load. Approximately 5 minutes.

1. Open <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce> and sign in. A free Kaggle account is required to download.
2. Click **Download**. The archive is about 45 MB compressed, roughly 120 MB unzipped. Unzip it into `data/raw/` inside the project folder.
3. Or, with the Kaggle CLI, after placing your API token at `~/.kaggle/kaggle.json` with permissions 600:

```
kaggle datasets download -d olistbr/brazilian-ecommerce \
  -p data/raw --unzip
```

Verify:

```
ls data/raw/*.csv | wc -l
```

Must print 9. The expected row counts, which the load reconciles against, are:

File	Rows	Feeds
<code>olist_orders_dataset.csv</code>	99,441	<code>fct_orders</code> , and every delivery measure
<code>olist_order_items_dataset.csv</code>	112,650	<code>fct_order_items</code> , the revenue backbone
<code>olist_order_payments_dataset.csv</code>	103,886	<code>fct_payments</code> (aggregated to order grain first)
<code>olist_order_reviews_dataset.csv</code>	99,224	the review score on <code>fct_orders</code> (deduplicated)
<code>olist_customers_dataset.csv</code>	99,441	<code>dim_customer</code> , keyed on the person
<code>olist_products_dataset.csv</code>	32,951	<code>dim_product</code>
<code>olist_sellers_dataset.csv</code>	3,095	<code>dim_seller</code>
<code>olist_geolocation_dataset.csv</code>	1,000,163	coordinates, collapsed to one row per zip prefix
<code>product_category_name_translation.csv</code>	71	English category names

`data/` is in `.gitignore`, so raw CSVs never enter the repository. Other members either download the file themselves or pull it from the GCS bucket after Step 6.

6. Step 5 — Bucket, Datasets and Config

Platform Owner creates the shared resources; every member does the config part. Approximately 10 minutes.

IRREVERSIBLE. A BigQuery dataset's location **cannot be changed after creation**, and a dataset in one location cannot be joined to a dataset in another. Everything here is **US multi-region**. The whole team must agree before this step runs; fixing it later means deleting and reloading everything.

6.1 Shared resources (Platform Owner only)

```
gcloud storage buckets create gs://<PROJECT_ID>-olist-raw \  
  --location=US --uniform-bucket-level-access
```

```
for ds in olist_raw olist_staging olist_marts olist_snapshots; do  
  bq --location=US mk --dataset "<PROJECT_ID>:$ds"  
done
```

Four datasets rather than one, so a defect stays reproducible in raw without contaminating what analysts read, and so the Analytics Owner can be given read-only access to marts alone.

6.2 Personal dev dataset (every member)

Everyone develops into their own dataset, so nobody can overwrite anyone else's tables. Only a passing build promotes anything to the shared marts.

```
bq --location=US mk --dataset "<PROJECT_ID>:dbt_<yourname>"
```

6.3 Config files (every member)

```
cp .env.example .env
```

Edit .env and fill in at least these three values:

```
GCP_PROJECT_ID=<your Project ID>  
GCS_RAW_BUCKET=<PROJECT_ID>-olist-raw  
DBT_DEV_DATASET=dbt_<yourname>
```

```
mkdir -p ~/.dbt  
cp dbt_project/profiles.yml.example ~/.dbt/profiles.yml
```

Both .env and ~/.dbt/profiles.yml are git-ignored. .env.example holds variable names only and is the file that is committed.

Verify the region, before any data is loaded:

```
bq show --format=prettyjson <PROJECT_ID>:olist_raw \  
  | grep location
```

Must print "location": "US". If it prints anything else, delete the dataset and recreate it **now**, while it is still empty.

Verify the dbt connection:

```
make debug
```

All connection checks must pass against your personal dbt_<yourname> dataset. Use `make debug`, **not** a bare `dbt debug`: the Makefile is the only thing that loads and exports .env, so a direct dbt command fails with *"Env var required but not provided"*.

7. Step 6 — Run the Pipeline

Anyone, any time. First run takes roughly 5 to 10 minutes, most of it uploading and loading.

```
make all
```

The stages run in order and the chain stops at the first failure:

manifest	Checksum the source files. sha256, byte size and the <i>true</i> CSV record count of all nine files. Review text contains embedded newlines, so a line count would be wrong; the loader parses the CSV properly.
GATE 1	Great Expectations, on the files, before the load. Expected files present, column set and order, row count against the manifest, non-null keys, money ranges, categorical domains. If this fails, nothing is loaded .
ingest	Upload and load. Uploads to the GCS landing prefix, loads into <code>olist_raw</code> with explicit schemas and no autodetect, then prints a source-versus-BigQuery row-count reconciliation table.
GATE 2	dbt build. 24 models, 1 snapshot and 161 tests, interleaved in dependency order so a failing test stops its own downstream models. If this fails, the marts do not publish .
unit	pytest. 19 tests over the ingestion helpers. Needs no cloud credentials.

7.1 Prove idempotency — this is a graded deliverable

Running the pipeline twice must produce identical row counts. The loader uses `WRITE_TRUNCATE` per batch, so a rerun replaces rather than appends. Demonstrating it is evidence, not decoration.

```
make ingest
```

```
bq query --use_legacy_sql=false \  
"SELECT COUNT(*) FROM \`<PROJECT_ID>.olist_raw.orders\`"
```

Must still return 99441 after the second run. Commit the reconciliation log to `docs/evidence/`.

7.2 Every make target

Command	What it does
<code>make help</code>	List every target with its description.
<code>make setup</code>	Install <code>requirements.txt</code> and the dbt packages. Once per machine.
<code>make manifest</code>	Checksum and row-count the local CSVs. Makes no cloud calls at all.
<code>make gate</code>	Quality Gate 1 on its own: validate the files without loading them.
<code>make ingest</code>	Upload to GCS, load into BigQuery, reconcile row counts.
<code>make build</code>	Quality Gate 2 on its own: dbt build (models, snapshot and tests).
<code>make test</code>	Run the dbt tests only, against whatever is already built.
<code>make unit</code>	Run the pytest suite. Needs no credentials.
<code>make lint</code>	sqlfluff over all dbt SQL.
<code>make all</code>	The whole pipeline: gate, ingest, build, unit.
<code>make analyse</code>	Execute all three notebooks top to bottom.
<code>make dashboard</code>	Serve the interactive dashboard.

Command	What it does
make docs	Generate and serve the dbt lineage graph.
make evidence	Collect test artifacts into docs/evidence/ for the report.
make clean	Remove dbt build artifacts.

8. Step 7 — Open the Dashboard

```
make dashboard
```

A browser opens at `http://localhost:8501`. If it does not open by itself, visit that address manually. Press `Ctrl+C` in the terminal to stop the server.

The sidebar filters every view by **purchase-month range**, **destination region**, and a **minimum order volume** below which a rate is treated as noise rather than a finding.

8.1 The five tabs

Tab	Answers	What to look at
Where	Which destinations and routes are late?	Late rate by destination state against the national average; an origin-to-destination region heat map; the routes carrying the most late orders, with the revenue behind each.
When	Which months, and is it improving?	Order volume against late rate by purchase month; a first-half versus second-half comparison stated in percentage points; and the decomposition showing whether days are lost in payment approval, seller handover or carrier transit.
What it costs	What does lateness cost us?	Review score against how early or late the order arrived; the same penalty recomputed <i>within</i> each state as a control; revenue exposed to late delivery; and whether the promised date itself is padded or optimistic.
Business KPIs	The three metrics the brief requires	Monthly sales with the late rate overlaid; a revenue-versus-late-rate scatter that surfaces categories which sell well but ship badly; RFM segments with their delivery experience attached.
Data quality	Can these numbers be trusted?	Runs the revenue reconciliation, the order-grain check, the person-grain RFM check and an app-versus-model agreement check live against the warehouse.

DEMONSTRATE THE DATA QUALITY TAB DURING THE PRESENTATION. It re-runs the reconciliations against the live warehouse in front of the audience. A dashboard that can prove its own numbers are not inflated by a join is far more persuasive than one that simply asserts it.

8.2 Why the dashboard cannot drift from the pipeline

- **It reads the marts layer only.** No CSV, no raw tables, no local files. Every number comes from a dbt model that has tests attached.
- **It does not redefine metrics.** Definitions live in `docs/metric_dictionary.md` and are implemented in dbt.
- Interactive filtering *does* force the late rate to be recomputed against `fct_orders`, because a pre-aggregated model cannot be sliced by an arbitrary filter combination. That SQL is confined to a single constant in the app, and the Data quality tab asserts live that it still agrees with `agg_delivery_monthly`.
- The shared connection module sits at the repository root and is imported by *both* the notebooks and the dashboard, so the two cannot end up reading different datasets while both appear correct.

9. Optional Outputs

9.1 Execute the notebooks

```
make analyse
```

Runs all three notebooks headless and writes the outputs back in place. This is exactly how a reviewer verifies reproducibility, so run it before submitting.

9.2 dbt documentation and lineage graph

```
make docs
```

Generates the model documentation and serves the full lineage graph in a browser. Useful as a slide during the technical part of the presentation.

9.3 Dagster orchestration UI

```
mkdir -p orchestration/.dagster_home  
export DAGSTER_HOME=$PWD/orchestration/.dagster_home  
dagster dev -f orchestration/definitions.py
```

Dagster requires DAGSTER_HOME to point at a directory that already exists; it does not create one for you, and fails with `is not a directory` or `does not exist` if the `mkdir` step above is skipped.

Opens at `http://localhost:3000`. This is where you demonstrate that a failing upstream quality gate **skips** the downstream assets, which is the difference between orchestration and a cron job that runs everything regardless.

10. Troubleshooting

Symptom	Cause and fix
Env var required but not provided: GCP_PROJECT_ID	<code>.env</code> does not exist, or GCP_PROJECT_ID is not filled in. The Makefile only includes <code>.env</code> when the file is present. Copy <code>.env.example</code> to <code>.env</code> and complete it.
could not automatically determine credentials	<code>gcloud auth application-default login</code> was skipped. It is a separate credential from <code>gcloud auth login</code> , and both are required.
Dataset <code>olist_raw</code> is in EU, but this project is pinned to US	The dataset was created in the wrong location. The loader asserts this and fails fast on purpose. Delete the dataset and recreate it with <code>--location=US</code> before loading anything.
Missing source file: <code>data/raw/olist_orders_dataset.csv</code>	The Kaggle archive was not unzipped into <code>data/raw/</code> . See Step 4.
Quality Gate 1 reports FAIL and nothing loads	This is the gate working as designed. The log names the file, the column and the number of offending records. Fix the source data and rerun. Do not bypass it.
A dbt test fails and the marts do not build	Also by design. Read the failing test name: the reconciliation tests in particular mean a join has fanned out and a revenue figure would have been wrong. Fix the model, do not lower the test.
dbt will not install, or reports a syntax error on startup	The wrong Python is active. Run <code>python --version</code> ; it must be 3.11. Run <code>conda activate bd-m2-g7</code> .
The dashboard shows "Not configured yet"	The app could not import the shared connection module because <code>.env</code> is missing or incomplete. It is showing setup instructions rather than a stack trace.
The dashboard shows "Could not reach the marts layer"	Credentials are fine but the marts do not exist yet. Run <code>make all</code> first.
Port 8501 is already in use	A dashboard is already running. Stop it with <code>Ctrl+C</code> in its terminal, or run <code>streamlit run dashboard/app.py --server.port 8502</code> .
<code>\$DAGSTER_HOME "..."</code> is not a directory or does not exist	Dagster requires the directory to already exist; it will not create one for you. Run <code>mkdir -p orchestration/.dagster_home</code> before the export.
Dagster: <code>profiles_dir ...</code> does not contain a <code>profiles.yml</code> file, thrown from inside <code>dagster_dbt/dbt_project.py</code>	A <code>dagster_dbt</code> internal step (<code>DbtProject.prepare_if_dev()</code>) builds its own <code>DbtCliResource</code> at import time and, unlike the dbt CLI itself, does not fall back to <code>DBT_PROFILES_DIR</code> or <code>~/dbt</code> . <code>orchestration/definitions.py</code> already passes <code>profiles_dir</code> directly to the <code>DbtProject(...)</code> constructor for this reason. If you see this error, something has reverted that argument — check it is still there before anything else.
Restarting <code>dagster dev</code> keeps showing the pre-fix error, or restarting seems to do nothing	<code>dagster dev</code> spawns separate <code>dagster_webserver</code> , <code>code-server</code> , <code>api_grpc</code> and <code>_daemon</code> child processes with their own command lines. Killing only the parent (<code>pkill -f "dagster dev"</code>) orphans the children, which keep the old, broken code location bound to port 3000. Stop everything with <code>pkill -9 -f dagster</code> and confirm the port is free (<code>lsof -ti :3000</code> returns nothing) before starting again.

11. What Has and Has Not Been Verified

Stated plainly so nobody is surprised on the first run.

11.1 Verified

Check	Result
dbt parse, clean project	Passes. 24 models, 161 tests, 1 snapshot resolve correctly.
sqlfluff, BigQuery dialect	Zero findings across all models and tests.
pytest	19 tests pass.
Dagster definitions load	28 assets, 1 job, 1 daily schedule.
Great Expectations gate	Correctly blocks and names the offending column and row count when faults are injected into a fixture.
Streamlit dashboard	All five tabs render correctly against a synthetic dataset, including every chart, table and the live reconciliation panel.
Secret and data leak guard	No <code>.env</code> , no credential JSON and nothing under <code>data/</code> would be tracked by git.

11.2 Not yet verified

THE SQL HAS NEVER RUN AGAINST A REAL BIGQUERY WAREHOUSE. At the time of writing there is no GCP project and no credentials, so every check above is *static*: the SQL was parsed by dbt and by sqlfluff against the BigQuery dialect, and the dashboard was exercised against synthetic data. Semantic correctness on the real dataset can only be confirmed once Step 6 has been run. **If dbt build reports an error on the first run, capture the full output before changing anything.**

11.3 Acceptance checklist for the first successful run

- ☐ `make all` completes with no failures.
- ☐ The reconciliation table printed by `make ingest` shows zero differences on all nine tables.
- ☐ Running `make ingest` a second time produces identical row counts.
- ☐ `dbt build` reports 0 failures across 161 tests.
- ☐ All three notebooks execute end to end with `make analyse`.
- ☐ `make dashboard` opens and the Data quality tab shows all four checks passing.
- ☐ `git status` shows no `.env`, no credential file and nothing under `data/`.